Creative Software Design
3. Lab2
C++: std::string
& Function Overloading

Dept. of Computer Science

Hanyang University

String

- C: C-style null-terminated string (using C-style array)
  - `char str1[] = "My String";`
  - Just an array of characters terminated with a null character ('\0')

- C++: **`std::string`**
  - `#include <string>`
  - `std::string str1 = "abc";`
  - `std::string str2("def");`
  - Many convenient operations are available such as:
    `str1 += "123" + str2.substr(0, 2);`
  - Much more powerful and convenient

    @see https://cplusplus.com/reference/string/string/

1. Word Counter for the Given File

- You can relay file contents into **`stdin`**

```
$ ./example01 < sample.txt

$ cat sample.txt | ./example01
```

- Compare your result with **`wc`**
  - **`wc`**: built-in program for counting words, lines, characters, and bytes

```
$ wc –w sample.txt
```


2. The Most Frequent Word in the File

- Words should be case-insensitive
- Compare your result with the following
  - built-in programs: cat, tr, sort, uniq
  - |: pipe operator
    - A | B ➔ A's **`stdout`** passed to B's **`stdin`**

```
$ cat sample.txt | tr ' ' '\n' | tr '[:upper:]' '[:lower:]' \
   | sort | uniq -c | sort -n
```

Function Overloading

- Functions handle different variables but perform similar tasks
- Instead of giving those functions different names
- Giving the same name with different function signatures
  - Function signature: name + variables
  - The compiler matches the caller to the callee function based on the function signature

```
int test1(int n, float m);        // different signatures,
double test1(float n, float m);   // hence allowed

int test2(int n, float m);      // the same signatures
double test2(int n, float m);   // compile error
```


3. Multiplicator

- Give the same function name to four different operators
  - two **`int`** numbers ➔ numeric multiplication
  - two **`double`** numbers ➔ numeric multiplication
  - two **`struct point`** ➔ pair-wise multiplication

```
struct point {
    int xpos;
    int ypos;
} Point;
```

  - two 2-D arrays or **`int`** ➔ pair-wise multiplication

Default Arguments

- **Default arguments** are parameters with its default value
  - Often called **optional parameters**

```
#include<iostream>
using namespace std;

int sum(int x, int y, int z = 0, int w = 0)
{
    return (x + y + z + w);
}

int main()
{
    cout << sum(10, 15) << endl;       // sum(10, 15, 0, 0)
    cout << sum(10, 15, 25) << endl;   // sum(10, 15, 25, 0)
    cout << sum(10, 15, 25, 30) << endl; // sum(10, 15, 25, 30)
    return 0;
}
```


4. Repeating Messages

- Define `repeat_msg()` function
  - Make the second parameter use a default argument

```
#include<iostream>

int main()
{
    repeat_msg("hi");
    repeat_msg("hello", 10);
    return 0;
}
```